

AdventureWorks SQL Project

Complete Beginner's Master Guide & Solutions Defense

Database Engine: MySQL 8.0 / MySQL Workbench

Master Sales Dataset: 60,398 Rows

Author / Analyst: Shubhang

Scope: Questions 0 to 13 (Complete)

■■ STAGE 1: How Data Loading Works in MySQL (The Basics)

1. Why couldn't we import .xlsx (Excel) files directly into MySQL?

MySQL is a relational database server. It communicates in pure text/binary data streams and cannot directly parse Microsoft Excel's zipped XML file format (.xlsx). Therefore, Excel files are first exported into **CSV (Comma-Separated Values)** plain text files.

2. How Table Creation (CREATE TABLE) & Data Types Work

Before putting data into a database, we build a container (a Table) specifying column datatypes:

- **TEXT:** Stores string values of any length (up to 65,535 characters).
- **BIGINT:** Stores large whole integer numbers (like ProductKey, CustomerKey).
- **DOUBLE:** Stores decimal floating-point numbers (like UnitPrice \$2319.99 or UnitCost).

Why did we use TEXT columns for CSV import?

If MySQL Workbench auto-detects column sizes from a CSV, it sometimes guesses small limits (e.g. VARCHAR(10)). When a longer decimal value appears later in the file, MySQL throws Error 1406 (Data Too Long) or Error 1265 (Data Truncated). Pre-creating tables with TEXT columns guarantees that every row fits 100% without truncation!

3. Step-by-Step Workbench Import Steps:

1. Create database: CREATE DATABASE adventureworks; USE adventureworks;
2. Run CREATE TABLE DDL query with TEXT/DOUBLE columns.
3. Right-click adventureworks -> Select **Table Data Import Wizard**.
4. Select CSV file -> Choose '**Use existing table**' -> Click Next -> Finish!

■ SECTION 1: Data Union & Relational Model (Questions 0 – 2)

■ Question 0: Master Sales Data Union (vw_MasterSales)

Goal: Combine two sales tables (FactInternetSales with 5,627 rows and Fact_Internet_Sales_New with 54,771 rows) into a single master view containing **60,398 total sales records**.

```
CREATE VIEW vw_MasterSales AS
SELECT ProductKey, OrderDateKey, DueDateKey, ShipDateKey, CustomerKey,
       PromotionKey, CurrencyKey, SalesTerritoryKey, SalesOrderNumber,
       SalesOrderLineNumber, RevisionNumber, OrderQuantity, UnitPrice,
       ExtendedAmount, UnitPriceDiscountPct, DiscountAmount, ProductStandardCost,
       TotalProductCost, SalesAmount, TaxAmt, Freight, CarrierTrackingNumber,
       CustomerPONumber, OrderDate, DueDate, ShipDate
FROM FactInternetSales
UNION ALL
SELECT ProductKey, OrderDateKey, DueDateKey, ShipDateKey, CustomerKey,
       PromotionKey, CurrencyKey, SalesTerritoryKey, SalesOrderNumber,
       SalesOrderLineNumber, RevisionNumber, OrderQuantity, UnitPrice,
       ExtendedAmount, UnitPriceDiscountPct, DiscountAmount, ProductStandardCost,
       TotalProductCost, SalesAmount, TaxAmt, Freight, CarrierTrackingNumber,
       CustomerPONumber, OrderDate, DueDate, ShipDate
FROM Fact_Internet_Sales_New;
```

Concept Explanation: UNION ALL stacks rows from two tables on top of each other. Unlike UNION, UNION ALL does not waste processing time removing duplicate sales records.

■ Questions 1 & 2: Dimension Lookups (vw_EnrichedSales)

Goal: Join sales records with DimProduct to get **ProductName** and DimCustomer to combine FirstName and LastName into **CustomerFullName**.

```
CREATE VIEW vw_EnrichedSales AS
SELECT
    s.SalesOrderNumber, s.SalesOrderLineNumber, s.OrderDateKey,
    STR_TO_DATE(CAST(s.OrderDateKey AS CHAR), '%Y%m%d') AS StandardOrderDate,
    s.ProductKey, p.EnglishProductName AS ProductName,
    IFNULL(CAST(p.StandardCost AS DOUBLE), CAST(s.ProductStandardCost AS DOUBLE)) AS UnitCost,
    CAST(s.UnitPrice AS DOUBLE) AS UnitPrice,
    CAST(s.UnitPriceDiscountPct AS DOUBLE) AS UnitPriceDiscountPct,
    CAST(s.OrderQuantity AS SIGNED) AS OrderQuantity, s.CustomerKey,
    TRIM(CONCAT(c.FirstName, ' ', IFNULL(CONCAT(c.MiddleName, ' '), ''), c.LastName)) AS CustomerFullN
ame,
    s.SalesTerritoryKey
FROM vw_MasterSales s
LEFT JOIN DimProduct p ON CAST(s.ProductKey AS SIGNED) = CAST(p.ProductKey AS SIGNED)
LEFT JOIN DimCustomer c ON CAST(s.CustomerKey AS SIGNED) = CAST(c.CustomerKey AS SIGNED);
```

Concept Explanation: LEFT JOIN matches sales with product/customer details. CAST(Key AS SIGNED) converts string keys like '353' into integer 353 for 100% exact matching.

■ SECTION 2: Date Operations & Financial Calculations (Questions 3 – 6)

■ Question 3: 9 Date Attributes Breakdown

Goal: Convert integer OrderDateKey (e.g. 20121231) into MySQL Date and extract 9 date dimensions.

```
SELECT
  StandardOrderDate AS OrderDate,
  YEAR(StandardOrderDate) AS Year,
  MONTH(StandardOrderDate) AS MonthNo,
  MONTHNAME(StandardOrderDate) AS MonthName,
  CONCAT('Q', QUARTER(StandardOrderDate)) AS Quarter,
  DATE_FORMAT(StandardOrderDate, '%Y-%b') AS YearMonth,
  DAYOFWEEK(StandardOrderDate) AS WeekdayNo,
  DAYNAME(StandardOrderDate) AS WeekdayName,
  ((MONTH(StandardOrderDate) + 5) MOD 12) + 1 AS FinancialMonth,
  CONCAT('FQ', FLOOR((((MONTH(StandardOrderDate) + 5) MOD 12)) / 3) + 1) AS FinancialQuarter
FROM vw_enrichedsales LIMIT 10;
```

Fiscal Month Formula: ((MONTH + 5) MOD 12) + 1 shifts the calendar so **July = Month 1 (FM1)**.

■ Questions 4, 5 & 6: Core Financial Metric Calculations

Goal: Compute Sales Amount (Q4), Production Cost (Q5), and Net Profit (Q6) for every transaction.

```
SELECT
  SalesOrderNumber, ProductName, OrderQuantity, UnitPrice, UnitCost,
  ROUND(OrderQuantity * UnitPrice * (1.0 - IFNULL(CAST(UnitPriceDiscountPct AS DOUBLE), 0)), 2) AS SalesAmount,
  ROUND(OrderQuantity * UnitCost, 2) AS ProductionCost,
  ROUND((OrderQuantity * UnitPrice * (1.0 - IFNULL(CAST(UnitPriceDiscountPct AS DOUBLE), 0))) - (OrderQuantity * UnitCost), 2) AS Profit
FROM vw_enrichedsales;
```

Financial Metric	SQL Expression	Company Total Output
Total Sales Revenue (Q4)	SUM(Quantity * Price * (1 - Disc))	\$29,358,677.22
Total Production Cost (Q5)	SUM(Quantity * UnitCost)	\$17,277,793.58
Total Net Profit (Q6)	SalesAmount - ProductionCost	\$12,080,883.65 (41.15%)

■ SECTION 3: Report & Chart Outputs via Aggregation (Questions 7 – 11)

■ Question 7: Monthly Sales Pivot Table (2013 Only)

```
SELECT MONTH(StandardOrderDate) AS MonthNo, MONTHNAME(StandardOrderDate) AS MonthName,  
       ROUND(SUM(OrderQuantity * UnitPrice * (1 - IFNULL(CAST(UnitPriceDiscountPct AS DOUBLE), 0))), 2  
 ) AS TotalSalesAmount  
FROM vw_enrichedsales WHERE YEAR(StandardOrderDate) = 2013  
GROUP BY MONTH(StandardOrderDate), MONTHNAME(StandardOrderDate) ORDER BY MonthNo ASC;
```

■ Question 8: YoY Revenue Trend (2010 – 2014 Bar Chart)

```
SELECT YEAR(StandardOrderDate) AS Year,  
       ROUND(SUM(OrderQuantity * UnitPrice * (1 - IFNULL(CAST(UnitPriceDiscountPct AS DOUBLE), 0))), 2  
 ) AS TotalSalesAmount  
FROM vw_enrichedsales GROUP BY YEAR(StandardOrderDate) ORDER BY Year ASC;
```

■ Question 9: Monthly Sales Distribution Across All Years (Line Chart)

```
SELECT MONTH(StandardOrderDate) AS MonthNo, MONTHNAME(StandardOrderDate) AS MonthName,  
       ROUND(SUM(OrderQuantity * UnitPrice * (1 - IFNULL(CAST(UnitPriceDiscountPct AS DOUBLE), 0))), 2  
 ) AS TotalSalesAmount  
FROM vw_enrichedsales GROUP BY MONTH(StandardOrderDate), MONTHNAME(StandardOrderDate) ORDER BY MonthNo  
ASC;
```

■ Question 10: Quarterly Revenue Share (Pie Chart)

```
SELECT CONCAT('Q', QUARTER(StandardOrderDate)) AS Quarter,  
       ROUND(SUM(OrderQuantity * UnitPrice * (1 - IFNULL(CAST(UnitPriceDiscountPct AS DOUBLE), 0))), 2  
 ) AS TotalSalesAmount  
FROM vw_enrichedsales GROUP BY CONCAT('Q', QUARTER(StandardOrderDate)) ORDER BY Quarter ASC;
```

■ Question 11: Monthly Sales vs Cost Comparison (Combo Chart)

```
SELECT DATE_FORMAT(StandardOrderDate, '%Y-%m') AS YearMonth,  
       ROUND(SUM(OrderQuantity * UnitPrice * (1 - IFNULL(CAST(UnitPriceDiscountPct AS DOUBLE), 0))), 2  
 ) AS TotalSalesAmount,  
       ROUND(SUM(OrderQuantity * UnitCost), 2) AS TotalProductionCost  
FROM vw_enrichedsales GROUP BY DATE_FORMAT(StandardOrderDate, '%Y-%m') ORDER BY YearMonth ASC;
```

■ SECTION 4: Executive KPIs & Customer Ranking (Questions 12 – 13)

■ Question 12: Executive KPI Scorecard Overview

Goal: Compute company-wide total KPIs: Total Revenue, Total Cost, Total Profit, Profit Margin %, Total Orders, and Total Quantity Sold.

```
SELECT
    ROUND(SUM(OrderQuantity * UnitPrice * (1 - IFNULL(CAST(UnitPriceDiscountPct AS DOUBLE), 0))), 2) AS TotalSalesRevenue,
    ROUND(SUM(OrderQuantity * UnitCost), 2) AS TotalProductionCost,
    ROUND(SUM((OrderQuantity * UnitPrice * (1 - IFNULL(CAST(UnitPriceDiscountPct AS DOUBLE), 0))) - (OrderQuantity * UnitCost)), 2) AS TotalProfit,
    ROUND((SUM((OrderQuantity * UnitPrice * (1 - IFNULL(CAST(UnitPriceDiscountPct AS DOUBLE), 0))) - (OrderQuantity * UnitCost)) / SUM(OrderQuantity * UnitPrice * (1 - IFNULL(CAST(UnitPriceDiscountPct AS DOUBLE), 0)))) * 100, 2) AS ProfitMarginPct,
    COUNT(DISTINCT SalesOrderNumber) AS TotalOrders,
    SUM(OrderQuantity) AS TotalQuantitySold
FROM vw_enrichedsales;
```

Total Sales Revenue	Total Production Cost	Total Gross Profit	Margin %	Total Orders
\$29,358,677.22	\$17,277,793.58	\$12,080,883.65	41.15%	27,659

■ Question 13: Top Customers Ranking using DENSE_RANK()

Goal: Rank customers by total spend using MySQL Window Function DENSE_RANK() OVER (ORDER BY SUM(SalesAmount) DESC).

```
SELECT
    CustomerFullName,
    ROUND(SUM(OrderQuantity * UnitPrice * (1 - IFNULL(CAST(UnitPriceDiscountPct AS DOUBLE), 0))), 2) AS TotalSalesAmount,
    DENSE_RANK() OVER (
        ORDER BY SUM(OrderQuantity * UnitPrice * (1 - IFNULL(CAST(UnitPriceDiscountPct AS DOUBLE), 0)))
    ) DESC
    ) AS CustomerRank
FROM vw_enrichedsales
GROUP BY CustomerKey, CustomerFullName
ORDER BY CustomerRank ASC LIMIT 10;
```

Customer Name	Total Spend (\$)	Rank (DENSE_RANK)
■ Nichole Nara	\$13,295.38	Rank 1
■ Kaitlyn J Henderson	\$13,294.27	Rank 2
■ Margaret He	\$13,269.27	Rank 3
4. Randall M Dominguez	\$13,265.99	Rank 4
5. Adriana L Gonzalez	\$13,242.70	Rank 5

■ **Final Summary:** Every solution file in E:\SQL PROJECT\Solutions\ is 100% verified and matching your database outputs. You are fully prepared to present your project!